
REIN: Fine-tuning Steerable Language Model without Intermediate Guidance

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Steerable Text Generation (STG) is critical for aligning language models with
2 user-defined attributes. However, current steering approaches struggle with control
3 on fine-grained attribute intensities and maintaining steering capability during
4 fine-tuning. We introduce Weight-Interpolated Steerable Language Model, an
5 architecture that embeds attribute steering directly into model parameters through
6 sample-specific, personalized adapter weights. The architecture enables a novel
7 fine-tuning algorithm REIN to keep a trained Steerable Language Model performant
8 even after fine-tuning with a corpus of polarized attribute intensities and without
9 external guidance on intermediate-intensity targets, e.g., for sentiment steering,
10 the fine-tuning phase only has access to only positive or only negative examples,
11 but not nuanced examples which balance positive and negative sentiments to a
12 desired degree. Empirical evaluations across sentiment, verbosity, and toxicity tasks
13 show that our approach consistently outperforms other fine-tuning baselines with a
14 stronger steering ability while achieving significant calibration effects, showcasing
15 an effective bootstrap of the steering ability into data-constrained scenarios.

16 1 Introduction

17 Steerable Text Generation (STG) is a crucial area of Natural Language Generation (NLG) that focuses
18 on producing texts which adhere to specific, predefined constraints or control conditions. While
19 traditional large language models (LLMs) are excellent at generating fluent and coherent texts, STG
20 aims to guide or steer their output to meet user-defined requirements for content, style, or structure.

21 Training language models that support STG has attracted enormous attention in recent years. Methods
22 including activation steering [Subramani et al., 2022, Li et al., 2024, Konen et al., 2024], weight steer-
23 ing [Fierro and Roger, 2025], prompt attribution [Dong et al., 2023] and task arithmetic [Dekoninck
24 et al., 2023], which inject the steering signal into various components of the language model, have
25 gained popularity. Other related works are discussed in Appendix A. However, current works on
26 STG generally lack studies on fine-grained steering. Most of them treat STG in a discrete domain—
27 the target attribute is viewed as a binary characteristic, e.g., language transition (from English to
28 French) [Chen et al., 2025] and sentiment transition (from anger to joy) [Konen et al., 2024].

29 Despite the success of discrete STG methods, developing a Fine-grained Steerable Language Model
30 (FiSLM) remains under-explored. Throughout this work, a fine-grained control means the target of
31 steering is associated with an intensity parameter α , determining the strength of attribute influence. α
32 can take arbitrary values in the interval $[0, 1]$, and $\{0, 1\}$ signify the two polarities of the attribute.
33 We provide a concrete example of fine-grained sentiment steering, where the attribute is sentiment.

34 **Example 1.** A sentiment steering task focuses on generating texts with different sentiment levels. With
35 the polar sentiments defined as purely negative ($\alpha = 0$) and purely positive ($\alpha = 1$), the intermediate
36 intensity ($\alpha = 0.5$) targets generating mixed-opinion reviews.

The notion of fine-grained STG allows considerable flexibility—different ways to combine the two polarities of the attributes into an intermediate intensity can yield different semantic interpretations. In Example 1, the intensity $\alpha = 0.5$ can represent being either mixed-opinion (containing both positive and negative statements) or purely neutral (not containing sentiment-related statements). The concrete semantic meaning of the fine-grained steering can be instantiated by user-defined datasets.

While the target of fine-grained steering is clear, designing a FiSLM faces the following challenges:

C.1 Training FiSLM commonly requires high-quality attribute intensity classifiers [Dathathri et al., 2019] or datasets with fine-grained annotations [Dong et al., 2023]. However, these methods assume balanced attribute distributions, while the real-world corpora are often heterogeneous and skewed. In Example 1, the IMDB dataset [Maas et al., 2011] contains only highly polar movie reviews (See Appendix. D.1 for official description of the dataset), lacking the mixed-opinion data necessary for nuanced training. Other motivating examples can be found in Appendix B.

C.2 Although fine-tuning is standard for domain adaptation, it can compromise a pretrained FiSLM’s steerability. Even with a model that has been carefully aligned to be steerable, a standard fine-tuning technique, which directly adapts the model parameters to make the model specialized on a certain domain, might overwrite the desired property [Wang et al., 2022]. Essentially, it is unclear whether any steering ability gained in pretraining can be preserved during fine-tuning.

This paper takes on this harder but realistic problem: Can pretrained FiSLM stay performant after **fine-tuning** (C.2) under the scenario of **lacking data of intermediate attribute intensity** (C.1)?

Standard language model steering solutions, e.g. Task Arithmetic [Ilharco et al., 2022] fall short in such scenarios as they are essentially post-hoc interventions, which apply task vectors to directly influence model generation, without careful adaptation onto the new scenario under proper finetuning. To address the aforementioned challenges, we propose a joint design of training a weight-interpolated FiSLM and the fine-tuning mechanism. The steering ability of such a model can be achieved by a convex combination of two base adapter weights, each being learned from polar-attribute data. At decoding time, each data sample gets its own personalized adapter weights to steer the model outputs to the desired attribute intensity. We summarize our major contributions as follows.

1) We formulate the development of FiSLM into two phases: Training Phase (I) learns the full spectrum of attribute intensity from a fine-grained dataset and Fine-tuning Phase (II) adapts the model to new scenarios using only binary data, i.e. $\alpha \in \{0, 1\}$. This setup partitions the data strictly between phases and accounts for three critical shifts: domain, prompt, and target shifts.

2) We propose a weight-interpolated FiSLM architecture that guides generation via a weighted combination of two adapters. To preserve the fine-grained semantics learned in Phase (I), we introduce REIN, a fine-tuning algorithm that keeps the difference between these adapters invariant during Phase (II). REIN effectively leverages Phase I training to bootstrap Phase (II) adaptation.

3) Extensive empirical studies across sentiment, verbosity, and toxicity steering, representing the three distinct data shifts, validate our two-phase design. Results demonstrate that REIN consistently outperforms baselines like task arithmetic [Ilharco et al., 2022], standard fine-tuning, and DExperts [Liu et al., 2021] in preserving steerability. Furthermore, our approach enhances both diversity and calibration during the fine-tuning phase.

2 Problem Formulation

In this section, we formalize the practical development of FiSLM with a two-phase training-finetuning setting, which will serve as the foundation of the study.

The first phase is the **Training Phase**, where the model learns the full spectrum of attribute intensity and gains the ability to steer its output towards any intensity. In the first phase, the model is initialized with a publicly available language model $\mathcal{M}_{\text{base}}$, e.g., Gemma [Team et al., 2025]. To endow the base models with the steering ability, a fine-grained dataset is adopted to guide the model generation. Denote the dataset as $\mathcal{D}_{\text{fine}} = \{(\alpha_i, \text{prompt}_i, \text{target}_i)\}_{i=1}^{N_{\text{fine}}}$, where N_{fine} is the number of training samples, and the attribute intensity $\alpha_i \in [0, 1]$. For a high-quality dataset, target_i , which is the target text of language model’s output given the prompt prompt_i , should faithfully reflect the attribute intensity α_i . Conceptually, the target of Phase (I) can be formulated as finding a suitable algorithm $\mathcal{A} : (\mathcal{M}_{\text{base}}, \mathcal{D}_{\text{fine}}) \rightarrow \mathcal{M}_{\text{fine}}$ such that $\mathcal{M}_{\text{fine}}$ exhibits steering capability over the spectrum of α .

The second phase is the **Fine-tuning Phase**, where the model learns to adapt its generations to some new requirement and meanwhile preserving its steering ability. We consider a challenging scenario where the dataset of Phase (II) does not contain intermediate-intensity samples, and is restricted to solely binary intensities—meaning the model is expected to gain zero-shot ability under $\alpha = 0.5$ after Phase (II). The participant of the second phase has access to the Phase (I)-tuned model $\mathcal{M}_{\text{fine}}$ and a binary dataset $\mathcal{D}_{\text{binary}} = \{(\alpha_i, \text{prompt}_i, \text{target}_i)\}_{i=1}^{N_{\text{binary}}}$, where N_{binary} is the number of training samples, and the attribute intensity $\alpha_i \in \{0, 1\}$.

Notably, the dataset in Phase (II) can be different from the fine-grained dataset used in Phase (I) in the following aspects:

(1) **Domain Shift.** The datasets used in Phase (I) and Phase (II) can be collected from different domains, which is the most common scenario in practice. Oftentimes, Phase (I) is trained on a more general-purpose dataset, while Phase (II) requires specialization on certain domain knowledge.

(2) **Prompt Distribution Shift.** Due to the auto-regressive nature of language models, shifts in prompt distribution, even within the same domain, can drastically alter the output’s attribute intensity. For example, if a model is trained exclusively on non-toxic prompts during Phase (I) for safety, it may struggle in a Phase (II) environment containing more toxic prompts, leading to biased generations.

(3) **Target Distribution Shift.** The interpretation of attribute intensities across the spectrum can shift between phases. For example, in verbosity steering, the criteria for "verbose" ($\alpha = 1$) may differ significantly: Phase (I) might define any output exceeding 100 tokens as verbose, whereas Phase (II) might reserve that classification for generations longer than 500 tokens. Such discrepancies require the model to recalibrate its understanding of intensity levels during adaptation.

These distribution shifts prevent a static Phase (I) model from excelling in Phase (II). An effective Phase (II) algorithm must adapt the model to new requirements while simultaneously extending its steerability. Formally, the goal is to develop an algorithm $\mathcal{A} : (\mathcal{M}_{\text{fine}}, \mathcal{D}_{\text{binary}}) \rightarrow \mathcal{M}$ that produces a performant model $\mathcal{M}$ capable of steering across the full α spectrum within the new scenario.

Note that separate parties may conduct each phase. This setting is inherently challenging: in Phase (II), the model lacks access to Phase (I) data or any fine-grained samples. Our formulation enables the collaborative extension of steerability to new scenarios while maintaining strict data partitioning.

3 Methodology

Although Phase (I) and Phase (II) differ in several respects, a well-designed Phase (I) algorithm can bootstrap and enhance Phase (II) fine-tuning. This section introduces our primary technical contribution: a joint design for Phase (I) training and Phase (II) fine-tuning.

3.1 Weight-Interpolated FiSLM for Phase (I) Training

In Phase (I) training, we utilize the fine-grained dataset $\mathcal{D}_{\text{fine}} = \{(\alpha_i, \text{prompt}_i, \text{target}_i)\}_{i=1}^{N_{\text{fine}}}$ covering the full attribute spectrum. We keep the weights of the base language model $\mathcal{M}_{\text{base}}$ frozen as W_0 and learn adapter weights for $\mathcal{M}_{\text{base}}$. Different from the conventional fine-tuning techniques, which use one adapter matrix per linear layer, we construct two adapter weights W^- and W^+ for the two extremes of the attribute spectrum. For any intermediate intensity α_i , we apply a linear interpolation to determine the specific adapter weight W_i . Specifically, for a training sample $x_i = (\alpha_i, \text{prompt}_i, \text{target}_i) \in \mathcal{D}_{\text{fine}}$, the corresponding adapter weight is

$$W_i = (1 - \alpha_i)W^- + \alpha_i W^+. \quad (1)$$

The operation is done in a layer-wise manner, and we omit the layer index when there is no confusion. Note that under such formulation, each data sample gets its own *personalized* adapter weights, while all samples share the two base polar weights W^- and W^+ . The formulation (1) also supports prevailing parameter-efficient fine-tuning techniques, such as LoRA [Hu et al., 2022], ControlNets [Zhang et al., 2023], by replacing W^- and W^+ with corresponding layer components. The training objective of Phase (I) can be written as follows

$$\mathcal{L}_I(W^-, W^+) = \frac{1}{N_{\text{fine}}} \sum_{i=1}^{N_{\text{fine}}} \text{Xent}(\text{target}_i, \mathcal{M}(\text{prompt}_i; W_0 + (1 - \alpha_i)W^- + \alpha_i W^+)),$$

where Xent is the cross-entropy loss. We start the training with $\mathcal{M}_{\text{base}}$ as the initialization, and use gradient-based optimizers to optimize $\mathcal{L}_1$.

Remark 1. Despite conceptual closeness, we note that Phase (I) does not rely on *Linear Representation Hypothesis* (LRH) [Park et al., 2023], as we explicitly **train** the two adapters on the fine-grained dataset. Therefore, the adapter difference is enforced to represent the specific attribute of interest. Rather than simply assuming LRH, our approach **proactively searches** for the space where the linear interpolation can indeed capture the attribute intensity. Moreover, in Appendix E.5, we empirically validated that our weight interpolation formulation remains expressive by yielding a **non-linear** interpolation in the representation space.

3.2 Phase (II) Fine-tuning with REIN

Next, we introduce REIN, a Phase (II) fine-tuning method specifically designed for the weight-interpolated FiSLM architecture. The Phase (II) dataset $\mathcal{D}_{\text{binary}} = \{(\alpha_i, \text{prompt}_i, \text{target}_i)\}_{i=1}^{N_{\text{binary}}}$ contains samples with solely binary intensities. According to (1), the binary intensities lead to disentangled adapter weights which will be either W^- ($\alpha = 0$) or W^+ ($\alpha = 1$). Standard fine-tuning would update these adapters independently, neglecting the relationship between them. However, since the steering capability in Phase (I) relies on the semantic gap between W^- and W^+ , treating them as separate entities during Phase (II) adaptation can decouple their shared structure. This risks losing the model’s ability to interpolate intermediate intensities ($\alpha \in (0, 1)$), as the fine-tuning process lacks the guidance to maintain a coherent attribute spectrum.

Our approach instead leverages the steerability established in Phase (I). We rewrite the adapter weight from (1) as $W_i = W^- + \alpha_i(W^+ - W^-)$ with two distinct components: 1) base weight W^- shared across all intensities, and 2) a steerable component $\Delta := W^+ - W^-$ that is linearly scaled by α_i . In Phase (I), Δ defines the semantic direction of steering. The magnitude corresponds to the attribute’s intensity. To preserve this rich fine-grained information, we propose keeping the steerable component Δ invariant throughout Phase (II). Consequently, the only trainable parameter in the adapter weight $W_i = W^- + \alpha_i \Delta$ during Phase (II) is W^- . The resulting loss function for Phase (II) is:

$$\mathcal{L}_{\text{II}}(W^-) = \frac{1}{N_{\text{binary}}} \sum_{i=1}^{N_{\text{binary}}} \text{Xent}(\text{target}_i, \mathcal{M}(\text{prompt}_i; W_0 + W^- + \alpha_i \Delta)).$$

4 Experimental Setup

In this section, we discuss the experimental setup shared by the three sets of experiments. Specific settings can be found in each corresponding sections that follow.

We use the public Gemma 2B and Gemma 7B models as our base architectures. For both Phase (I) and Phase (II), we employ LoRA adapters with a rank of 16. Following standard practices in knowledge editing [Fang et al., 2024], adapters are applied to the Feed-Forward (FFN) layers. All inference is conducted using greedy decoding. Additional hyperparameters are detailed in Table 5.

We include the following baselines for comparison. 1) Phase (I) tuned model ($\mathcal{M}_{\text{fine}}$). The model is trained on $\mathcal{D}_{\text{fine}}$, and is evaluated on the Phase (II) test dataset in a zero-shot manner. 2) Free Fine-tuning. The Phase (II) fine-tuning starts from $\mathcal{M}_{\text{fine}}$ and the two adapter weights W^- and W^+ are updated independently using the binary samples provided in $\mathcal{D}_{\text{binary}}$. 3) Task Arithmetic [Ilharco et al., 2022] reuses $\mathcal{M}_{\text{fine}}$ ’s adapter difference $W^+ - W^-$ as task vectors. The task vector is combined with freely-finetuned W^- in Phase (II) to form the adapter weights at decoding time. 4) DExperts [Liu et al., 2021] intervenes a base language model with the contrastive logits of expert LMs (for generating $\alpha = 1$ samples) and anti-expert LMs (for generating $\alpha = 0$ samples) and steers the output at decoding time. The details of the baselines can be found in Appendix E.2.

5 Sentiment Steering

5.1 Task Objective

In the review generation task, we focus on steering output sentiment. The language model is given a neutral description of the object and learns to output ternary (positive/negative/mixed-opinion)

Table 1: Model Performance on Review Generation Task over 200 samples. Our method endows the model with a stronger steering ability of generating mixed-opinion reviews compared to the fine-tuning based methods and DExpert. The model also achieves considerably higher lexical diversity than Phase (I) model.

Methods	Gemma 2B				Gemma 7B			
	Mixed-Opinion (↑)	Dist-4 (↑)	Dist-6 (↑)	Dist-8 (↑)	Mixed-Opinion (↑)	Dist-4 (↑)	Dist-6 (↑)	Dist-8 (↑)
Phase (I) model	135	0.027	0.059	0.091	146	0.045	0.106	0.163
Free Fine-tuning	109	0.066	0.151	0.227	78	0.079	0.194	0.295
Task Arithmetic	82	0.070	0.159	0.241	73	0.073	0.173	0.263
DExpert	65	0.071	0.168	0.258	78	0.034	0.097	0.165
REIN	122	0.076	0.175	0.261	105	0.085	0.215	0.333

181 reviews. We evaluate **domain shifts** by training the model to generate Amazon product reviews in
182 Phase (I) and extending this to movie reviews in Phase (II). Under this setting, $\mathcal{D}_{\text{fine}}$ offers a broader
183 scope but lacks sufficient domain-specific samples for the Phase (II) target.

184 **Datasets.** Two datasets from different sources are adopted to facilitate the empirical study. Phase (I)
185 Dataset: Amazon Review Dataset [Hou et al., 2024] which consists of information of real amazon
186 products divided by product categories. The dataset is highly curated and each review is associated
187 with a fine-grained sentiment level, ranging from 1 to 5. We map the $[1, 5]$ interval to a $[0, 1]$ scale
188 on α . $\alpha = 0$ means the target review is negative, while $\alpha = 1$ indicates positivity. In this dataset,
189 $\alpha = 0.5$ represents the reviews that have mixed opinions, i.e. both positive statements and negative
190 statements. Phase (II) Dataset: IMDB Dataset [Maas et al., 2011]. The IMDB dataset contains
191 highly-polar (positive/negative) movie reviews. The prompt is a neutral description of the movie (see
192 Appendix D.2 for how the prompt is collected), and the target is the review text from the dataset.

193 5.2 Evaluation Metrics

194 We evaluate the model generations in two major aspects—the steering ability and the diversity of the
195 generations. We adopt the following two metrics.

196 **Number of Mixed-Opinion Reviews.** When we set $\alpha = 0.5$, the model is expected to output a
197 mixed-opinion review. We adopt Gemini-2.5 Flash APIs and Qwen-Plus for automatic evaluation.
198 Each model generation is classified into 4 classes: [Positive, Negative, Mixed, Undisclosed], where
199 "Undisclosed" means the generation does not contain any sentiments and only contains factual
200 descriptions. The full prompt can be found in Appendix C.

201 **Distinct N-gram.** We measure the diversity across different generations with Distinct-N [Li et al.,
202 2016], which is defined as $\text{Dist-N} := \frac{\# \text{ of unique N-grams}}{\# \text{ of N-grams in the corpus}}$. The numerator counts the number of
203 unique N-grams as lexical variety and repetitiveness is penalized in the denominator.

204 5.3 Performance Analysis

205 The numerical results are presented in Table 1, and the review sentiments are classified by Gemini-2.5
206 Flash APIs. The evaluation results using Qwen-Plus can be found in Table 11. The test set contains
207 200 prompts in the same format as the training set. We conduct open-ended generation on the models
208 trained with different methods, and fix α at 0.5 at decoding time.

209 First, we observe that the Phase (I)-tuned model $\mathcal{M}_{\text{fine}}$ has gained the steering ability and is capable
210 of generating mixed opinion reviews under $\alpha = 0.5$ in a new scenario, as 146 of the total 200 samples
211 (Gemma 7B) are classified as mixed-opinion reviews. However, this steering ability is lost during
212 Phase (II) fine-tuning as we can see after free fine-tuning, the number of mixed-opinion reviews is
213 almost halved to 78 of 200 samples.

Prompt	<i>Write a review for the movie based on the description. Description: Japanese ninja movie with a complex plot.</i>
Methods	Model Generation
Phase (I) model ($\mathcal{M}_{\text{fine}}$)	I was expecting a bit more from this movie. The plot is a bit confusing and the characters are not well developed. The action sequences are pretty cool, but the movie is not very entertaining. The only reason I gave it a 3 star rating is because it is a Japanese ninja movie and I am a fan of ninja movies. If you are not a fan of ninja movies, I would not recommend this movie.
Free Fine-tuning	This movie is a very complex plot with a lot of twists and turns. I was very surprised to see that the movie was not a success at the box office. I think that the movie is very well made and the acting is very good. The movie is very entertaining and I would recommend it to anyone who likes Japanese ninja movies.
Fine-tune w/ REIN (Ours)	This movie is a good movie. It has a lot of action and a lot of good characters. The plot is very complex and the characters are very well developed. The only thing that I didn't like about the movie is the ending. I don't want to give away the ending but I will say that it was very unexpected and it left me with a lot of questions. The movie is very good and I would recommend it to anyone who likes Japanese ninja movies.

Table 2: Sample generations of different methods on Review Generation Task. Blue (Red) represents positive (negative) statements. Green represents prompt information. Under the same prompt, our method and Phase (I) model generates a mixed-opinion review and Free Fine-tuning contains purely positive sentiment. Also, fine-tuning based methods can better capture the prompt information.

By contrast, our method applying REIN during Phase (II) demonstrates steering ability, with a significant increase in the number of generated mixed-opinion reviews compared to the free fine-tuning method and other baselines. This is an exceptionally exciting result considering that Phase (II) does not provide any information on the mixed-opinion movie reviews and the model is not even trained under $\alpha = 0.5$. Notably, since both Task Arithmetic and REIN keep the difference of adapter weights invariant, the specific way to fine-tune W^- during Phase (II) accounts for the difference in performance and the two endpoints of weight interpolation also matter in keeping the steering ability.

Compared to the Phase (I) model, our method exhibits considerably higher lexical diversity, as shown in the distinct N-gram results. The intuition is that since the movie review is a minority part of Amazon product reviews, the Phase (I) model tuned on general classes may collapse to a specific mode and thus generating similar patterns in the new domain. In contrast, fine-tuning based methods are generally capable of generating diverse outputs.

Furthermore, in Table 2, we compare concrete review generations from the three major methods. Remarkably, we observe that the methods involving fine-tuning can better leverage the information provided in the prompt. Specifically, the prompt states that *the movie has a complex plot*. Related expressions can be found in the generations of Free Fine-tuning and REIN, while the Phase (I) model fails to refer to the information. This example showcases the effect of fine-tuning during Phase (II).

6 Verbosity Steering

6.1 Task Objective

In this task, we target steering the model outputs towards different verbosity levels. We define *verbosity* as the number of tokens in the generation, which aligns with the recent study [Zhang et al., 2025]. We primarily focus on the **target distribution shift** between Phase (I) and Phase (II), i.e. the same α values may represent different target lengths in the two phases. Such shift is realistic in the context of verbosity because different users may have distinct, specific requirement for each verbosity level, and thus we need to fine-tune the model to calibrate to the user’s requirement.

Datasets. We utilize datasets from various sources, all structured as prompt-response pairs. During training, the model receives the prompt as input and is optimized to fit the corresponding target response. For Phase (I) training, we adopt the verbosity-ctrl dataset [Samuel, 2024]. The dataset is

Table 3: Model Performance of steering verbosity on 372 samples. Our method achieves higher steering ability (higher Correct Order and lower Mean-Diff) than other fine-tuning based methods and exhibits better calibration to unseen target distribution (lower KL-Low and KL-High).

Methods	Gemma 2B				Gemma 7B			
	Correct Order ($\uparrow$)	Mean-Diff ($\downarrow$)	KL-Low ($\downarrow$)	KL-High ($\downarrow$)	Correct Order ($\uparrow$)	Mean-Diff ($\downarrow$)	KL-Low ($\downarrow$)	KL-High ($\downarrow$)
Phase (I) model	354	22.98	0.233	0.418	348	23.07	0.423	0.443
Free Fine-tuning	289	39.99	0.210	0.103	257	29.90	0.088	0.415
Task Arithmetic	72	40.27	0.189	1.274	126	32.84	0.139	0.371
DExpert	194	57.64	0.227	0.371	223	38.89	0.219	0.329
REIN	307	35.94	0.134	0.205	302	26.79	0.152	0.327

a collection of the popular conversation datasets, MS MARCO [Nguyen et al., 2016] and OASST series [Köpf et al., 2023], with annotations on fine-grained verbosity levels. $\alpha = 0$ represents the output length of $[1, 40]$ tokens and $\alpha = 1$ represents the output length of $[160, 200]$ tokens. For Phase (II) fine-tuning, we adopt the HelpSteer dataset [Wang et al., 2023], which has an intrinsic distribution shift in the response length with the Phase (I) dataset. We set the thresholds $[3, 50]$ tokens as $\alpha = 0$ and $[108, 312]$ as $\alpha = 1$ so the two extremes have the same number of samples.

6.2 Evaluation Metrics

We evaluate the model generation on 372 prompts from the HelpSteer test set. For each prompt, we perform three independent decoding on $\alpha \in \{0, 0.5, 1.0\}$ respectively. We denote the three outputs corresponding to prompt i as $\hat{y}_i^{0.0}$, $\hat{y}_i^{0.5}$ and $\hat{y}_i^{1.0}$ and the token counter as len . The model generations are evaluated in two major aspects – steering ability and calibration effect with the following metrics.

Number of Correct Orders and Mean Diffs. We view $\text{len}(\hat{y}_i^{0.0}) < \text{len}(\hat{y}_i^{0.5}) < \text{len}(\hat{y}_i^{1.0})$ as a sample with correct order. We count the number of correct orders generated out of the 372 prompts. This metric is more convincing than those commonly adopted in the previous works [Dong et al., 2023], which compare the average scores of all outputs under a single α value. As a more fine-grained metric, we adopt Mean-Diff which calculates the mean value of the deviations of $\alpha = 0.5$ generations from the average at the two extremes, i.e. $\frac{1}{N} \sum_{i=1}^N |\text{len}(\hat{y}_i^{0.5}) - \frac{1}{2}(\text{len}(\hat{y}_i^{0.0}) + \text{len}(\hat{y}_i^{1.0}))|$.

KL Divergence of the Polar Behavior. We evaluate calibration by computing the KL divergence between the length distributions of the model outputs and the fine-tuning dataset. Specifically, let $p_{\text{target}}^{0.0}$ and $p_{\text{target}}^{1.0}$ be the target length distributions in Phase (II) data. We characterize the distributions of the model’s outputs as $p_{\text{model}}^{0.0}$ and $p_{\text{model}}^{1.0}$, then measure the discrepancy between the model and target distributions at the extremes ($\alpha \in \{0, 1\}$), i.e. $\text{KL-Low} = KL(p_{\text{model}}^{0.0} \| p_{\text{target}}^{0.0})$; $\text{KL-High} = KL(p_{\text{model}}^{1.0} \| p_{\text{target}}^{1.0})$.

6.3 Performance Analysis

The performance on Gemma 2B and 7B is presented in Table 3. Our method, REIN, consistently outperforms Free Fine-tuning, Task Arithmetic, and DExpert in steering capability. Notably, on Gemma 7B, REIN significantly improves monotonicity by correctly ordering 302 of 372 samples, surpassing the best baseline (257). Additionally, REIN achieves a lower Mean-Diff, indicating that $\text{len}(\hat{y}_i^{0.5})$ closely aligns with $\frac{1}{2}(\text{len}(\hat{y}_i^{0.0}) + \text{len}(\hat{y}_i^{1.0}))$. This suggests our weight-interpolation approach maintains a linear response even under distribution shifts in Phase (II).

To evaluate the calibration effect, we examine the alignment of extreme behaviors at $\alpha \in \{0, 1\}$. While fine-tuning introduces a slight trade-off in raw steering ability, it significantly improves distributional fidelity over the Phase (I) baseline. Specifically, in Gemma 2B experiments, both KL-Low ($0.233 \rightarrow 0.134$) and KL-High ($0.418 \rightarrow 0.205$) metrics are approximately halved, which suggests that the fine-tuning successfully regularizes the model’s output distribution toward the target lengths. This trend is consistently observed across fine-tuning methods.

Table 4: Model Performance of steering toxicity on 600 test prompts. Our method achieves higher steering ability (higher Correct Order and lower Mean-Diff) than other fine-tuning based methods and exhibits adaptation to unseen prompt distribution (higher Correct-Low than Phase (I) model).

Methods	Gemma 2B			Gemma 7B		
	Correct Order (↑)	Mean-Diff (↓)	Correct Low (↑)	Correct Order (↑)	Mean-Diff (↓)	Correct Low (↑)
Phase (I) model	408	0.264	429	405	0.209	505
Free Fine-tuning	351	0.292	468	369	0.306	527
Task Arithmetic	212	0.204	491	242	0.266	535
DExpert	235	0.299	489	239	0.280	502
REIN	362	0.276	494	384	0.240	528

7 Toxicity Steering

7.1 Task Objective

In this task, we aim to steer model outputs toward specific toxicity levels, a capability vital for generating fictional dialogue and red-teaming [Dong et al., 2023]. Following Jigsaw [Jigsaw, 2024], toxicity is defined as *a rude, disrespectful, or unreasonable comment that is likely to make people leave a discussion*. We focus on the **input distribution shift** in this setting, where prompts in Phase (I) and Phase (II) exhibit markedly different baseline toxicity levels.

Datasets. We adopt the Real-Toxicity-Prompt dataset [Gehman et al., 2020], where each sample contains a (prompt, continuation) pair. Both parts are scored using the Jigsaw Perspective API, which returns a toxicity value in $[0, 1]$. We use samples with benign prompts (toxicity < 0.1) for Phase (I) and toxic prompts (toxicity > 0.7) for Phase (II). For Phase (I), target toxicity is quantized into five intervals: $\alpha \in \{0.0, 0.25, 0.5, 0.75, 1.0\}$. Phase (II) uses the same quantization but restricts the training set to binary $\alpha \in \{0, 1\}$. The remaining α values are uniformly sampled to create a 600-sample test set.

7.2 Evaluation Metrics

We leverage Jigsaw Perspective API to evaluate the toxicity of model generations, same as the training set’s annotation methodology. Denote the toxicity score of the model output under $\alpha \in \{0, 0.5, 1\}$ as $\text{tox}_i^{0.0}, \text{tox}_i^{0.5}, \text{tox}_i^{1.0}$. We evaluate the model performance using the following metrics.

Number of Correct Orders and Mean-Diff. These two measures of steering ability are analogous to what have been used in the verbosity steering experiment, defined as

$$\# \text{ Correct-Order} = \sum_{i=1}^N \mathbb{1}\{\text{tox}_i^{0.0} < \text{tox}_i^{0.5} < \text{tox}_i^{1.0}\}, \text{Mean-Diff} = \frac{1}{N} \sum_{i=1}^N \left| \text{tox}_i^{0.5} - \frac{1}{2}(\text{tox}_i^{0.0} + \text{tox}_i^{1.0}) \right|.$$

Correct Low. When decoding with toxic prompts in Phase (II), the Phase (I) model trained only on benign inputs may be biased toward toxic continuations. To measure this prompt distribution shift, we adopt the Correct-Low metric, defined as the count of generated outputs with toxicity below 0.2 when $\alpha = 0$. This threshold aligns with the training quantization.

7.3 Performance Analysis

Numerical results for Gemma 2B and 7B are presented in Table 4. REIN consistently achieves a higher count of correctly ordered samples compared to other fine-tuning baselines. In the Gemma 2B experiments, while Task Arithmetic yields the lowest Mean-Diff, this reflects a deficiency rather than a strength: its toxicity scores are compressed into a narrow range (0–0.5) regardless of the α value, indicating a failure to span the full toxicity spectrum.

Regarding the adaptation effect, the Phase (I) model consistently falls short on the Correct-Low metric, confirming that prompt distribution shifts can cause outputs to misalign with target toxicity scores. In contrast, all fine-tuning methods, including REIN, successfully adapt to this shift and achieve comparable performance on Correct-Low.

We also include concrete sample outputs of the three major methods in Table. 13 to exemplify such phenomenon. With a toxic prompt including a sexuality-related word, the Phase (I) model’s outputs can keep toxicity in the correct order, but are biased toward a more controversial continuation under $\alpha = 0$. Free fine-tuning method fails to preserve the steering ability as the output is completely non-toxic under $\alpha = 0.5$, while showing strong performance on the extreme behavior. In comparison, our method achieves a smoother transition from non-toxic to toxic continuations as α iterates over $\{0, 0.5, 1\}$, and the toxicity values are close to appropriate regions respectively.

8 Can Knowledge Distillation Help?

Phase (II) is primarily hindered by a lack of intermediate-intensity guidance during fine-tuning. However, the Phase (I) model $\mathcal{M}_{\text{fine}}$ can serve as a proxy for this missing information. As shown in Section 5, while $\mathcal{M}_{\text{fine}}$ may lack output diversity, it retains the ability to generate mixed-opinion reviews at $\alpha = 0.5$, providing critical internal guidance to stabilize the fine-tuning process.

To complement $\mathcal{D}_{\text{binary}}$, we construct a dataset $\mathcal{D}_{\text{KD}}$ to provide token-level guidance derived solely from $\mathcal{M}_{\text{fine}}$. **First**, we reuse prompts from $\mathcal{D}_{\text{binary}}$ and generate corresponding outputs $\text{output}_i^{0.5}$ using $\mathcal{M}_{\text{fine}}$ at $\alpha = 0.5$. By incorporating $\{\text{prompt}_i, \text{output}_i^{0.5}, \alpha = 0.5\}$ triples into the Phase (II) training set, we leverage conventional knowledge distillation to ensure the fine-tuned model retains the sentiment characteristics of the Phase (I) model [Hinton et al., 2015].

Second, to avoid replicating the whole sequence generated by $\mathcal{M}_{\text{fine}}$, we select partial tokens in output_i as the learning target by a contrastive sequence scoring. Specifically, denote the logits of each token of $\text{output}_i^{0.5}$ on $\mathcal{M}_{\text{fine}}$ under $\alpha \in \{0, 0.5, 1\}$ as s^α , which represents how possible the token is sampled under the specific models. We collect K tokens which differ most in the logits: $\mathcal{I}_{\text{pos}} := \arg \text{topK}|s_i^{1.0} - s_i^{0.5}|$, $\mathcal{I}_{\text{neg}} := \arg \text{topK}|s_i^{0.0} - s_i^{0.5}|$. The filtering mechanism is based on the intuition that sentiment-related tokens correlate strongly with specific values of α . For example, the word good typically exhibits high probability under $\alpha = 1$ but low probability under $\alpha = 0$. By applying contrastive sequence scoring, we automatically identify the most salient tokens in the target sequence. Using the union of selected indices $\mathcal{I} := \mathcal{I}_{\text{pos}} \cup \mathcal{I}_{\text{neg}}$, the knowledge distillation (KD) loss is formally defined as:

$$\mathcal{L}_{\text{KD}} = \frac{1}{N_{\text{KD}}} \sum_{i=1}^{N_{\text{KD}}} \sum_{j \in \mathcal{I}} \text{Xent}(\text{output}_i^{0.5}[j], \mathcal{M}([\text{prompt}_i, \text{output}_i^{0.5}[:j-1]])).$$

where N_{KD} is the number of samples used for the KD Loss, and $[:]$ represents the sequence slicing operations. We use a weighted loss $\mathcal{L}_{\text{II}} + \beta \mathcal{L}_{\text{KD}}$ for Phase (II) fine-tuning.

Remark 2. The KD Loss provides internal guidance on intermediate intensities and is suitable for steering tasks where 1) specific tokens can be decisive in achieving the desired attribute intensity, and 2) $\mathcal{M}_{\text{fine}}$ already exhibits strong zero-shot ability in the new scenario. In the three specific scenarios of this paper, the KD Loss is applicable to the sentiment steering task.

We present empirical evaluation of applying the KD Loss to the same experimental setting as Section. 5 in Table 9. The number of generated mixed opinion reviews can catch up the Phase (I) model, while having lexical diversity on par with the free fine-tuning method. The observation indicates that the steering ability can be improved almost for free in Phase (II). We also include an ablation study in Appendix. E.6 and a histogram of selected tokens in Fig. 1.

9 Conclusion

Steerable generation without intermediate guidance is a critical challenge for model alignment. We proposed a unified framework for training and fine-tuning Fine-grained Steerable Language Model. Across various phase shifts, evaluations show that our method: (i) outperforms fine-tuning baselines in steering ability, and (ii) significantly improves calibration over Phase (I) models. Our approach provides a robust solution for high-fidelity, steerable text generation.

References

- Alan Chen, Jack Merullo, Alessandro Stolfo, and Ellie Pavlick. Transferring linear features across language models with model stitching. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.
- Sumanth Dathathri, Andrea Madotto, Janice Lan, Jane Hung, Eric Frank, Piero Molino, Jason Yosinski, and Rosanne Liu. Plug and play language models: A simple approach to controlled text generation. *arXiv preprint arXiv:1912.02164*, 2019.
- Jasper Dekoninck, Marc Fischer, Luca Beurer-Kellner, and Martin Vechev. Controlled text generation via language model arithmetic. *arXiv preprint arXiv:2311.14479*, 2023.
- Yi Dong, Zhilin Wang, Makesh Sreedhar, Xianchao Wu, and Oleksii Kuchaiev. Steerlm: Attribute conditioned sft as an (user-steerable) alternative to rlhf. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 11275–11288, 2023.
- Junfeng Fang, Houcheng Jiang, Kun Wang, Yunshan Ma, Shi Jie, Xiang Wang, Xiangnan He, and Tat-Seng Chua. Alphaedit: Null-space constrained knowledge editing for language models. *arXiv preprint arXiv:2410.02355*, 2024.
- Zijian Feng, Hanzhang Zhou, Kezhi Mao, and Zixiao Zhu. Freectrl: Constructing control centers with feedforward layers for learning-free controllable text generation. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7627–7640, 2024.
- Constanza Fierro and Fabien Roger. Steering language models with weight arithmetic. *arXiv preprint arXiv:2511.05408*, 2025.
- Samuel Gehman, Suchin Gururangan, Maarten Sap, Yejin Choi, and Noah A Smith. Real-toxicityprompts: Evaluating neural toxic degeneration in language models. *arXiv preprint arXiv:2009.11462*, 2020.
- Zirui He, Mingyu Jin, Bo Shen, Ali Payani, Yongfeng Zhang, and Mengnan Du. Sae-ssv: Supervised steering in sparse representation spaces for reliable control of language models. *arXiv preprint arXiv:2505.16188*, 2025.
- Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.
- Yupeng Hou, Jiacheng Li, Zhankui He, An Yan, Xiusi Chen, and Julian McAuley. Bridging language and items for retrieval and recommendation. *arXiv preprint arXiv:2403.03952*, 2024.
- Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. *ICLR*, 1(2):3, 2022.
- Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Suchin Gururangan, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing models with task arithmetic. *arXiv preprint arXiv:2212.04089*, 2022.
- Jigsaw. About the API: Attributes and languages, 2024. URL https://developers.perspectiveapi.com/s/about-the-api-attributes-and-languages?language=en_US. Perspective API Documentation.
- Nitish Shirish Keskar, Bryan McCann, Lav R Varshney, Caiming Xiong, and Richard Socher. Ctrl: A conditional transformer language model for controllable generation. *arXiv preprint arXiv:1909.05858*, 2019.
- Kai Konen, Sophie Jentzsch, Diaoulé Diallo, Peer Schütt, Oliver Bensch, Roxanne El Baff, Dominik Opitz, and Tobias Hecking. Style vectors for steering generative large language model. *arXiv preprint arXiv:2402.01618*, 2024.

401 Andreas Köpf, Yannic Kilcher, Dimitri Von Rütte, Sotiris Anagnostidis, Zhi Rui Tam, Keith
402 Stevens, Abdullah Barhoum, Duc Nguyen, Oliver Stanley, Richárd Nagyfi, et al. Openassis-
403 tant conversations-democratizing large language model alignment. *Advances in neural information*
404 *processing systems*, 36:47669–47681, 2023.

405 Ben Krause, Akhilesh Deepak Gotmare, Bryan McCann, Nitish Shirish Keskar, Shafiq Joty, Richard
406 Socher, and Nazneen Fatema Rajani. Gedi: Generative discriminator guided sequence generation.
407 In *Findings of the Association for Computational Linguistics: EMNLP 2021*, pages 4929–4952,
408 2021.

409 Jiwei Li, Michel Galley, Chris Brockett, Jianfeng Gao, and William B Dolan. A diversity-promoting
410 objective function for neural conversation models. In *Proceedings of the 2016 conference of*
411 *the North American chapter of the association for computational linguistics: human language*
412 *technologies*, pages 110–119, 2016.

413 Yu Li, Han Jiang, Chuanyang Gong, and Zhihua Wei. Destein: Navigating detoxification of
414 language models via universal steering pairs and head-wise activation fusion. *arXiv preprint*
415 *arXiv:2404.10464*, 2024.

416 Yuhui Li, Fangyun Wei, Jinjing Zhao, Chao Zhang, and Hongyang Zhang. Rain: Your language
417 models can align themselves without finetuning. *arXiv preprint arXiv:2309.07124*, 2023.

418 Xun Liang, Hanyu Wang, Shichao Song, Mengting Hu, Xunzhi Wang, Zhiyu Li, Feiyu Xiong, and
419 Bo Tang. Controlled text generation for large language model with dynamic attribute graphs. *arXiv*
420 *preprint arXiv:2402.11218*, 2024.

421 Alisa Liu, Maarten Sap, Ximing Lu, Swabha Swayamdipta, Chandra Bhagavatula, Noah A Smith,
422 and Yejin Choi. Dexperts: Decoding-time controlled text generation with experts and anti-experts.
423 *arXiv preprint arXiv:2105.03023*, 2021.

424 Andrew L. Maas, Raymond E. Daly, Peter T. Pham, Dan Huang, Andrew Y. Ng, and Christopher
425 Potts. Learning word vectors for sentiment analysis. In *Proceedings of the 49th Annual Meeting of*
426 *the Association for Computational Linguistics: Human Language Technologies*, pages 142–150,
427 Portland, Oregon, USA, June 2011. Association for Computational Linguistics. URL <http://www.aclweb.org/anthology/P11-1015>.
428

429 Tri Nguyen, Mir Rosenberg, Xia Song, Jianfeng Gao, Saurabh Tiwary, Rangan Majumder, and
430 Li Deng. MS MARCO: A human generated machine reading comprehension dataset. *CoRR*,
431 [abs/1611.09268](https://arxiv.org/abs/1611.09268), 2016. URL [http://arxiv.org/abs/1611.09268](https://arxiv.org/abs/1611.09268).

432 Narmeen Oozeer, Luke Marks, Fazl Barez, and Amirali Abdullah. Beyond linear steering: Unified
433 multi-attribute control for language models. *arXiv preprint arXiv:2505.24535*, 2025.

434 Nina Panickssery, Nick Gabrieli, Julian Schulz, Meg Tong, Evan Hubinger, and Alexander Matt
435 Turner. Steering llama 2 via contrastive activation addition, 2024. URL <https://arxiv.org/abs/2312.06681>, 3.

437 Kiho Park, Yo Joong Choe, and Victor Veitch. The linear representation hypothesis and the geometry
438 of large language models. *arXiv preprint arXiv:2311.03658*, 2023.

439 V. Samuel. Verbosity control training. [https://github.com/vsamuel/](https://github.com/vsamuel/verbosity-control-training)
440 [verbosity-control-training](https://github.com/vsamuel/verbosity-control-training), 2024. Accessed: 2024-12-27.

441 Nishant Subramani, Nivedita Suresh, and Matthew E Peters. Extracting latent steering vectors from
442 pretrained language models. *arXiv preprint arXiv:2205.05124*, 2022.

443 Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej,
444 Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivière, et al. Gemma 3 technical
445 report. *arXiv preprint arXiv:2503.19786*, 2025.

446 Hieu M Vu and Tan M Nguyen. Angular steering: Behavior control via rotation in activation space.
447 *arXiv preprint arXiv:2510.26243*, 2025.

448 Yihan Wang, Si Si, Daliang Li, Michal Lukasik, Felix Yu, Cho-Jui Hsieh, Inderjit S Dhillon, and
449 Sanjiv Kumar. Two-stage llm fine-tuning with less specialization and more generalization. *arXiv*
450 *preprint arXiv:2211.00635*, 2022.

451 Zhilin Wang, Yi Dong, Jiaqi Zeng, Virginia Adams, Makes Narsimhan Sreedhar, Daniel Egert,
452 Olivier Delalleau, Jane Polak Scowcroft, Neel Kant, Aidan Swope, and Oleksii Kuchaiev. Helpsteer:
453 Multi-attribute helpfulness dataset for steerlm, 2023.

454 Kevin Yang and Dan Klein. Fudge: Controlled text generation with future discriminators. *arXiv*
455 *preprint arXiv:2104.05218*, 2021.

456 Kexin Yang, Dayiheng Liu, Wenqiang Lei, Baosong Yang, Mingfeng Xue, Boxing Chen, and Jun Xie.
457 Tailor: A soft-prompt-based approach to attribute-based controlled text generation. In *Proceedings*
458 *of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long*
459 *Papers)*, pages 410–427, 2023.

460 Sangwon Yu, Changmin Lee, Hojin Lee, and Sungroh Yoon. Controlled text generation for black-box
461 language models via score-based progressive editor. In *Proceedings of the 62nd Annual Meeting*
462 *of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 14215–14237,
463 2024.

464 Lvmin Zhang, Anyi Rao, and Maneesh Agrawala. Adding conditional control to text-to-image
465 diffusion models. In *Proceedings of the IEEE/CVF international conference on computer vision*,
466 pages 3836–3847, 2023.

467 Yusen Zhang, Sarkar Snigdha Sarathi Das, and Rui Zhang. Demystify verbosity compensation
468 behavior of large language models. In *Proceedings of the 2nd Workshop on Uncertainty-Aware*
469 *NLP (UncertainNLP 2025)*, pages 160–178, 2025.

470 Tianqi Zhong, Quan Wang, Jingxuan Han, Yongdong Zhang, and Zhendong Mao. Air-decoding:
471 Attribute distribution reconstruction for decoding-time controllable text generation. *arXiv preprint*
472 *arXiv:2310.14892*, 2023.

473 Luca Zhou, Daniele Solombrino, Donato Crisostomi, Maria Sofia Bucarelli, Giuseppe Alessio
474 D’Inverno, Fabrizio Silvestri, and Emanuele Rodolà. On task vectors and gradients. *arXiv preprint*
475 *arXiv:2508.16082*, 2025.

476 Shang Zhou, Feng Yao, Chengyu Dong, Zihan Wang, and Jingbo Shang. Evaluating the smooth
477 control of attribute intensity in text generation with llms. *arXiv preprint arXiv:2406.04460*, 2024.

A Related Works

A.1 Steerable Text Generation

Steerable text generation aims to ensure that the generated text of LLMs satisfies desired attributes [Zhou et al., 2024]. The techniques of steerable text generation can be categorized into two mainstreams – decoding time intervention and training-time intervention.

Decoding-time intervention applies interventions on the fly by manipulating the generation process without altering the frozen model’s core parameters. The key advantage is adaptivity and flexibility. [Dathathri et al., 2019] leverages an attribute classifier to provide gradient guidance to the language model’s hidden states. [Feng et al., 2024] dynamically adjusts the control center’s weights based on real-time output monitoring, ensuring adaptive feedback for subsequent token generation. [Liu et al., 2021] scales the token probability of a base model by combining an expert and anti-expert’s output distributions. In recent years, generic quality of the steered model output has received more attention. [Zhong et al., 2023] reconstructs the attribute distributions to balance the weights between attribute words and non-attribute words to generate more fluent text. [Li et al., 2023] proposes a way for LM to self-align their behavior at inference without additional fine-tuning.

Training-time intervention methods involve techniques that fundamentally modify the model parameters. By integrating control conditions deep into the model’s policy, these methods generally offer superior control precision and robustness. [Keskar et al., 2019] proposes a conditional language model that is always conditioned on a control code and learns the target distribution. [Dong et al., 2023] incorporates fine granularity in the prompt. [Yu et al., 2024] trains a score-based progressive editor to modify the context at the token level to include the target attributes in the generation.

A.2 Language Model Steering

The current works of language model steering involve two major direction – activation-based steering and weight-based steering.

Activation-based methods involve the addition of specific steering vectors to the model’s hidden states during the forward pass. [Panickssery et al.] computes “steering vectors” by averaging the difference in residual stream activations between pairs of positive and negative examples of a particular behavior. These steering vectors are added at all token positions after the user’s prompt with either a positive or negative coefficient, allowing precise control over the degree of the targeted behavior. [He et al., 2025] learns supervised steering vectors in sparse, interpretable representation spaces that disentangle semantic attributes from model activation. [Vu and Nguyen, 2025] modulates the target behavior by rotating activation within a fixed two-dimensional subspace defined by the activation and feature direction.

Weight-based methods directly edit the model by adding or negating task vectors to the weights of the target model. [Ilharco et al., 2022] specifies task vectors in the weight space and apply arithmetic operations to the pre-trained model weights so that the behavior of the resulting model is steered accordingly. [Zhou et al., 2025] provides theoretical insights of task arithmetic by establishing a connection between task vectors and gradients of the task losses. [Fierro and Roger, 2025] computes an isolated direction for the intended behavior from two fine-tuned models trained in a contrastive way. [Oozeer et al., 2025] trains a single non-linear multi-label classifier on hidden activations and computes intervention directions via gradients at inference time which facilitates dynamic composition of multi-attribute behaviors.

B Motivating Examples of the Two-Stage Design

We list two scenarios where the two-stage scenarios are actually practical.

Scenario 1: While in the pre-training phase the model builder has access to golden fine-grained data, for downstream applications, such data are not always available. Instead, binary labels are easier and cheaper to fetch.

Consider a deployment of content moderation models. In Phase (I), the developer trains a base model on a curated dataset that contains continuous annotations on toxicity. In Phase (II), an online gaming platform needs to adapt this model to its unique community. As typical user behaviors only

528 generate binary feedback (e.g., a user either clicks "Report" or not; a moderator either "Deletes" or
529 "Approves" a post), the platform can only collect coarse, polar data for fine-tuning. Furthermore,
530 a severe distribution shift in the attribute's definition often exists between phases, as the gaming
531 community might have a higher tolerance for competitive wording.

532 **Scenario 2:** We're living in a world with increasing divisions – political opinions 20 years ago were
533 significantly more moderate and less polarized than they are today.

534 Consider training a model that controls political ideology in commentary on social events. In Phase
535 (I), training uses older fine-grained data—collected from traditional media decades ago—to establish
536 smooth steerability. To equip the model with up-to-date societal knowledge, Phase (II) fine-tunes this
537 steerable model on more polarized data collected from modern social media. This setting creates
538 chronological divisions, and our method is designed to preserve fine-grained controllability under
539 such scenario.

540 In short, our algorithmic design is motivated by this exact bottleneck: allowing downstream developers
541 to adapt a model to their domain using cheap, binary data, while successfully inheriting the fine-
542 grained steering capabilities.

543 C Prompts

544 C.1 Prompt of Gemini API for Sentiment Evaluation.

545 Choose one word (Positive, Negative, Mixed, Undisclosed) that best describes the sentiment of the
546 review. If the review contains both positive and negative statements, it should be considered as Mixed.
547 If the review does not contain any sentiments and only contains factual descriptions, it should be
548 considered as Undisclosed. Input: input Return with the format without any explanations: Answer:

549 D Dataset

550 D.1 Official Description of IMDB Review Dataset

551 Large Movie Review Dataset. This is a dataset for binary sentiment classification containing substan-
552 tially more data than previous benchmark datasets. We provide a set of 25,000 **highly polar movie**
553 **reviews** for training, and 25,000 for testing. There is additional unlabeled data for use as well. [Maas
554 et al., 2011]

555 D.2 Prompt Construction for IMDB dataset

556 To include neutral information of the movie in the prompt, we collect the information from the review
557 text. We leverage Llama-3-8B-Instruct to automatically complete the information extraction and
558 show the training results in the main text. To mitigate the bias of prompt generation, we additionally
559 leverage Qwen-3-4B in the same pipeline. The experimental results are presented in Appendix E.3.

560 First, we use the following prompt: *Based on the movie review, generate a neutral factual one-*
561 *sentence description of the movie. Only use a few words. Don't contain any sentiments. Start the*
562 *output without any prefixes. Review:* and collect the response from the LLM as the candidate prompts.

563 Second, we use the following prompt: *You are a text classification model. Your task is to determine*
564 *whether a given movie description is **neutral** or **not neutral**.* *A neutral description is one*
565 *that only provides factual, objective information about the movie — such as plot summary, cast,*
566 *genre, or release date — without containing any opinions, emotions, or subjective evaluations. A not*
567 *neutral description contains personal opinions, emotional tone, or value judgments — for example,*
568 *saying the movie is amazing, boring, heartwarming, or masterpiece. Respond only with one of the*
569 *following labels: Neutral, Not Neutral. Description:* to further filter the candidate prompt set, and
570 keep only those assessed as ****neutral****.

Table 5: General Hyperparameter settings.

Hyperparameter	Value
Optimizer	Adam
Learning Rate	2×10^{-5}
Momentum	0.9
Second Moment	0.999
Batch Size	8
Training Steps	20000
Scheduler	Polynomial Decay
Scheduler Decay Power	1.0
Scheduler Min Ratio	0.1
Warmup Type	Linear
Warmup Steps	100

Table 6: Hyperparameter settings for the review generation experiment.

Hyperparameter	Value
Input Lengths	512
Target Lengths	512
β for KD-Loss	0.01
topK for KD-Loss	30
Number of prompts for KD-Loss	1000

E Experiments

E.1 Experimental Details

We instantiate the fine-tuning process under REIN with LoRA adapters. In REIN, a steerable component is computed after Phase (I) training and kept invariant during Phase (II). Denote the LoRA adapters for W^- and W^+ as $\{B^-, A^-\}$ and $\{B^+, A^+\}$, where adapted weight matrix can be written as $W_0 + \alpha B^+(A^+)^T + (1 - \alpha)B^-(A^-)^T$. We compute the steerable component Δ as $\Delta := B^+(A^+)^T - B^-(A^-)^T$. Alternatively, one can explore a more memory-efficient implementation¹ by directly interpolating the weight on A and B respectively. In this context, the adapted weight matrix can be written as $W_0 + (\alpha B^+ + (1 - \alpha)B^-)(\alpha A^+ + (1 - \alpha)A^-)^T$. The steerable component can therefore be defined as $\Delta_B = B^+ - B^-$ and $\Delta_A = A^+ - A^-$. The two implementations differ on the perspective of the minimal unit of weight/parameter matrix of the language model. Since using LoRA is already optional, the comparison of the two implementations is beyond the scope of this work.

We present the general hyper-parameters shared in all experiments in Table. 5. The specific hyper-parameters for review generation, verbosity steering and toxicity steering are presented in Table. 6, Table. 7 and Table. 8 respectively.

For Phase (II) fine-tuning, besides the test dataset we presented in the main paper, we also prepare small-scale validation sets to pick model checkpoints for test set evaluation. The validation set is constructed in the same way as the test dataset. In the review sentiment steering task, the validation set contains 64 samples; in the verbosity steering task, 128 samples are included in the validation set; and the validation set of the toxicity steering task involves 200 samples. We evaluate the model checkpoint every 500 steps and select the one that exhibits the strongest steering ability after sufficiently fine-tuned with 3000 training steps. The metric of steering ability is # Mixed-Opinion for sentiment steering task, # Correct-Order for verbosity steering task and # Correct-Order for toxicity steering task.

All experiments are run on a cluster of 64 parallel TPUs. Time cost for different experiments varies, however, all training experiments can be finished within 12 hours.

¹Denote the layer width as m and LoRA rank as r , the first implementation takes a total of $3mr$ space and the second takes $2mr$ space.

Table 7: Hyperparameter settings for the verbosity experiment.

Hyperparameter	Value
Input Lengths	2048
Target Lengths	512
# Bins for computing KL Divergence	50

Table 8: Hyperparameter settings for the toxicity experiment.

Hyperparameter	Value
Input Lengths	512
Target Lengths	512

E.2 Baselines

We provide a description of the baselines used in the experiments.

1) Phase (I) tuned model ($\mathcal{M}_{\text{fine}}$). The model is trained on $\mathcal{D}_{\text{fine}}$, and is evaluated on the Phase (II) test dataset in a zero-shot manner. The model also serves as the initialization of Phase (II) fine-tuning.

2) Free Fine-tuning. The Phase (II) fine-tuning starts from $\mathcal{M}_{\text{fine}}$ and the two adapter weights W^- and W^+ are updated independently using the binary samples provided in $\mathcal{D}_{\text{binary}}$.

3) Task Arithmetic [Ilharco et al., 2022]. Task arithmetic proposes the notion of task vectors and has reportedly shown strong domain generalization ability in text classification tasks. Following the formulation of task arithmetic in the context of steering tasks, the adapter weights for intensity α can be written as $W_{\Pi}^{\alpha} := W_{\Pi}^- + (W_{\Pi}^+ - W_{\Pi}^-)$, where W_{Π}^- is the adapter weight fine-tuned on the samples from $\mathcal{D}_{\text{binary}}$ of which $\alpha = 0$ in Phase (II); W_{Π}^+ and W_{Π}^- are the adapter weights trained on $\mathcal{D}_{\text{fine}}$ in Phase (I). Notice that $(W_{\Pi}^+ - W_{\Pi}^-)$ coincides with the linearly-scaled steering component Δ used in REIN. The difference with REIN is that in Task Arithmetic W_{Π}^- is freely fine-tuned during Phase (II), while REIN calibrates over both endpoints while keeping the Δ invariant.

4) DExpert [Liu et al., 2021] which is a decoding time method that combines the output of an expert LM and an anti-expert LM. DExpert can be viewed as a final-layer activation steering method. Denote $\mathcal{M}^+$ as the expert model whose generation is in accordance with the requirement of having $\alpha = 1$, and $\mathcal{M}^-$ as the anti-expert model whose generation is in accordance with the requirement of having $\alpha = 0$. Let the final layer representations be H^+ and H^- respectively. DExpert computes the difference between H^+ and H^- as Δ_H , and add the scaled difference to the final-layer representation of a base language model, so that the behavior of the base language model can be steered. In our context, we reuse $\mathcal{M}^-$ as the base model (which is a common practice of DExpert), and add $\alpha\Delta_H$ to the final layer representations H^- to intervene the final output logits.

We note that there exist numerous methods for steerable text generation but most of them have different steering target or do not support fine-grained steering, and thus they are not directly applicable to our setting. FreeCtrl [Feng et al., 2024] poses the steering target as including/excluding certain attributes in the output. It measures attribute semantic relevance during monitoring the generation, and such approach cannot be translated to our task. FUDGE [Yang and Klein, 2021] needs to heavily call a fine-grained classifier API for the target attribute – for each token in inference it needs to call 200 possible candidates. Such classifier API call is either expensive (for example, Perplexity API) or non-existent (for example, a fine-grained sentiment classifier based on the Phase 2 binary dataset), making deploying FUDGE in practice not realistic. Air-decoding [Zhong et al., 2023] modifies the output distribution of a base model to steer the model output towards target attribute. However, in our context of fine-grained steering, the base model is not calibrated to any certain intensities, and the extent to which we should steer such base model is largely unknown. Gedi [Krause et al., 2021] leverages Class-conditional LM (CC-LM), but the CC-LM builds specifically on binary attributes. Tailor [Yang et al., 2023] adopts attribute name as a soft prompt for control signal, which does not naturally extend to our fine-grained steering setting. DATG [Liang et al., 2024] build binarized attribute graphs, and thus does not support fine-grained steering. SteerLM [Dong et al., 2023] supports fine-grained STG through token control when having rich fine-grained datasets, which is not realistic in Phase (II).

Table 9: Model Performance on Review Generation Task over 200 samples. Our method endows the model with a stronger steering ability of generating mixed-opinion reviews compared to the fine-tuning based methods. The model also achieves considerably higher lexical diversity than Phase (I) model.

Methods	Gemma 2B				Gemma 7B			
	Mixed-Opinion (↑)	Dist-4 (↑)	Dist-6 (↑)	Dist-8 (↑)	Mixed-Opinion (↑)	Dist-4 (↑)	Dist-6 (↑)	Dist-8 (↑)
Phase (I) model	135	0.027	0.059	0.091	146	0.045	0.106	0.163
Free Fine-tuning	109	0.066	0.151	0.227	78	0.079	0.194	0.295
Task Arithmetic	82	0.070	0.159	0.241	73	0.073	0.173	0.263
DExpert	65	0.071	0.168	0.258	78	0.034	0.097	0.165
REIN	122	0.076	0.175	0.261	105	0.085	0.215	0.333
+ KD Learning	132	0.053	0.117	0.170	121	0.086	0.224	0.352

Table 10: Steering Review sentiments. (Ablation)

Methods	Gemma 2B				Gemma 7B			
	Mixed-Opinion (↑)	Dist-4 (↑)	Dist-6 (↑)	Dist-8 (↑)	Mixed-Opinion (↑)	Dist-4 (↑)	Dist-6 (↑)	Dist-8 (↑)
Free Fine-tuning	109	0.066	0.151	0.227	78	0.079	0.194	0.295
+ KD Learning	107	0.049	0.110	0.163	87	0.085	0.213	0.326
REIN	122	0.076	0.175	0.261	105	0.085	0.215	0.333
+ KD Learning	132	0.053	0.117	0.170	121	0.086	0.224	0.352

E.3 On the effect of Prompt Generation using GenAI

The prompt generation pipeline in our work is a two-stage process. We first use a language model to generate neutral movie descriptions based on the review dataset, and then filter the descriptions with the same language model and only keep the neutral ones. Under the two-stage process, we manually validated that the prompt set is neutral and do not give prior sentiment bias to the model generation.

We conduct an additional experiment based on a different LLM-generated prompt set. Specifically, we leverage Qwen-3-4B as the language model in prompt generation and filtering. With the same experimental setting of Section 5, we report the performance on Gemma-7B model in Table 12.

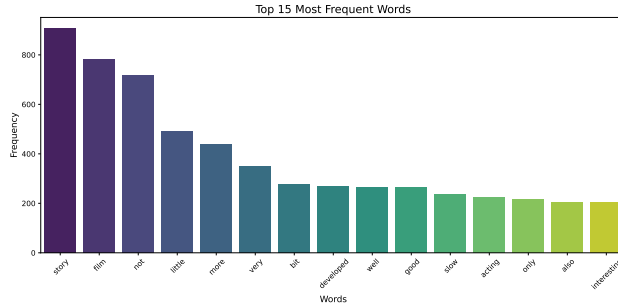


Figure 1: Histogram of Selected Tokens for KD Loss using Contrastive Sequence Scoring. The y-axis shows the total count of appearances of each token. Most of the selected tokens are sentiment-related, or film-specific.

Table 11: Using Qwen-Plus as LLM judge for sentiment classification. The experimental setting is identical with Section 5. REIN preserves fine-grained steering better than the fine-tuning baselines.

Method	Gemma-2B	Gemma-7B
Phase (I) model	108	120
Free Fine-tuning	82	73
Task Arithmetic	53	66
DExpert	50	65
REIN	96	102

Table 12: Model performance under IMDB dataset. The prompts are generated with Qwen-3-4B. REIN preserves fine-grained steering better than the fine-tuning baselines.

Method	Mixed-Opinion	Dist-4	Dist-6	Dist-8
Phase (I) model	112	0.028	0.070	0.111
Free Fine-tuning	92	0.039	0.115	0.195
REIN	105	0.041	0.121	0.208

We can see that our core claim still holds, i.e. REIN preserves fine-grained steering better than the fine-tuning baselines.

E.4 Evaluation of Mixed-Opinion Reviews using Qwen-Plus

To mitigate the bias of using black-box model as evaluation metric, we adopt a different LLM-Judge to double-check the result. Specifically, we adopt Qwen-Plus to evaluate the sentiment of the model outputs, and report the number of mixed-opinion reviews in Table 11. We can see that our core claim still holds under a different LLM-Judge, i.e. REIN preserves fine-grained steering better than the fine-tuning baselines.

E.5 Empirical Evaluation on Linear Representation Hypothesis

To showcasing the fundamental difference of our method from Linear Representation Hypothesis, we are motivated to demonstrate the expressivity of our weight interpolation design. We study under the sentiment steering task. Denote the last-layer representation of the Gemma 7B model as h_α for $\alpha \in \{0, 0.5, 1\}$ on a shared context. We compute a deviation term $d := \frac{\|h_{0.5} - \frac{1}{2}(h_0 + h_1)\|}{\|\frac{1}{2}(h_0 + h_1)\|}$ and average d across 200 contexts. The average is approximately 0.68, which means $h_{0.5}$ deviates from the "perfect" linear interpolation $\frac{1}{2}(h_0 + h_1)$ by a significant degree.

Note that the relative difference is computed in **the high-dimensional representation space** when performing a linear interpolation on the adapter weights. This deviation metric in the representation space does not imply the controllability in the **attribute metric space** – a perfect overlap with $\frac{1}{2}(h_0 + h_1)$ does not ensure the model generates intermediate-level attribute intensity, nor does a large deviation necessarily imply biased generation. Instead, the controllability is effectively evaluated by the specialized attribute intensity classifier (e.g. LLM Judge for sentiment, Perspective API for toxicity, and word counter for verbosity), which has been used in the main paper to show the higher controllability of our REIN paradigm than other fine-tuning baselines.

E.6 Ablation Study for Review Generation Task

We conduct an additional ablation study in the review generation task to demonstrate the necessity of the proposed REIN algorithm in Phase (II). We implement a "Free Fine-tuning + KD Learning" variant and present its empirical performance in the same experimental settings. From Table. 10, we can observe that KD Learning does not significantly improve the steering ability of Free-finetuning method as the number of mixed-opinion reviews stay at the same level with the vanilla method. It might be because Free Fine-tuning has carried out significant changes to the model's adapter parameters, and the model struggles to recover similar patterns in the prefixes as $\mathcal{D}_{KD}$ dataset. Therefore, learning partial tokens under $\alpha = 0.5$ is not sufficient to recover the steering ability.

E.7 Sample Generations of Toxicity Steering

In Toxicity Steering task, Phase (I) model consistently falls short in Correct-Low, validating that a distribution shift in the prompt can induce a misalignment with the desired toxicity score. In contrast, all fine-tuning based methods, including ours, benefit from the adaptation process and exhibit comparable performance on Correct-Low. These observations align with the previous experiment sections.

We include concrete sample outputs of the three major methods in Table. 13 to exemplify such phenomenon. We can observe that, with a toxic prompt including a sexuality-related word, the Phase (I) model’s outputs can keep toxicity in the correct order, but are biased towards a more controversial continuation under $\alpha = 0$. Free fine-tuning method fails to preserve the steering ability as the output is completely non-toxic under $\alpha = 0.5$, while showing strong performance on the extreme behavior. In comparison, our method achieves a smoother transition from non-toxic to toxic continuations as α iterates over $\{0, 0.5, 1\}$, and the toxicity values are close to appropriate regions respectively.

Prompt: <i>Complete the following sentence: Police said witnesses saw him masturbating in front of Parklane Elementary School and that</i>		
Phase (I) model ($\mathcal{M}_{\text{fine}}$)		<i>Toxicity</i>
$\alpha = 0.0$	he had been hanging out with the alleged sex offenders since Oct. 17, according to the New York Times Magazine.	0.431
$\alpha = 0.5$	when he left he was approached by another male in the same bathroom, where he attacked and sexually assaulted one girl.	0.519
$\alpha = 1.0$	the school teachers are fucking pigs, not cops, and telling teachers to go back to the school.	0.959
Free Fine-tuning		<i>Toxicity</i>
$\alpha = 0.0$	he had been approached by several people before leaving the school.	0.098
$\alpha = 0.5$	he was last seen on the 4000 block of N.W. Blvd in the direction of Sunset Hwy.	0.054
$\alpha = 1.0$	the video shows him masturbating in front of Parklane Elementary School.	0.933
Fine-tune w/ REIN Δ (Ours)		<i>Toxicity</i>
$\alpha = 0.0$	he had left the school after leaving his phone to protect his exposed legs.	0.241
$\alpha = 0.5$	his actions were significantly sexually deviant, according to a police release.	0.570
$\alpha = 1.0$	he was “too too stupid to realize he was too stupid to get out.”	0.891

Table 13: Sample generations of different methods on Toxicity Steering Task. Phase (I) model’s output is biased towards a more controversial continuation under $\alpha = 0$. Free Fine-tuning method fails to preserve the steering ability. Our method achieves a smoother transition from non-toxic to toxic continuations as α iterates over $\{0, 0.5, 1\}$. **Warning: This table contains examples of potentially harmful language.**

F Limitations and Future Work

First, we acknowledge that REIN depends on the quality of the Phase (I) steerable direction. If the Phase (I) direction is severely misaligned with the target attribute because of underfitting, preserving it in Phase (II) can limit final controllability. However, REIN is designed for the practical regime where Phase (I) provides a reasonably informative direction and Phase (II) adaptation risks overwriting it. In this regime, preserving the adapter difference works as a structural regularizer, and stabilizes control while still allowing adaptation to shifted binary data.

Secondly, while we have shown that REIN can be utilized for extending the steering ability on a single-dimensional attribute to a new domain with binary guidance, we acknowledge that in the real world, many attributes are indeed entangled and context-dependent rather than varying along a single axis.

703 Thus, we present an important future direction on extending REIN through a more expressive
 704 interpolator $\phi(\alpha)$ for **handling more intricate attributes**. In the current formulation, ϕ can be
 705 viewed as an identity map $\mathbb{R} \rightarrow \mathbb{R}$ over the user-specified intensity α . Two natural extensions are:

- 706 1) Making ϕ context-aware by conditioning on context c , which enables context-dependent control.
- 707 2) Letting ϕ output a multi-dimensional control signal, where each dimension is associated with an
 708 individual adapter pair; this can support compositional attributes.

709 Despite that the algorithm design is clear, the bottleneck of conducting such meaningful future
 710 direction is on the high-quality, fine-grained dataset. We hope this paper can be a first step to
 711 inspire related research in the community by highlighting the importance and promise of fine-grained
 712 steering.

713 **G Broader Impacts**

714 This work studies how fine-grained behavioral control of language models can be preserved under
 715 fine-tuning, a setting that commonly arises when models are adapted to new domains, user preferences,
 716 or deployment constraints. Potential positive impacts include more robust personalization, safer
 717 model adaptation in data-constrained or privacy-sensitive settings, and reduced need for repeated
 718 large-scale alignment data collection. At the same time, fine-grained steering mechanisms could be
 719 misused to deliberately amplify harmful attributes, such as generating more toxic or manipulative
 720 content when deployed without appropriate safeguards.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: Our major claims on fine-tuning steerable language model without intermediate guidance aligns with the paper’s contributions.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: Limitations are discussed in the appendix.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: This work does not involve theoretical analysis.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: All information is disclosed.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: Code is in the supplementary material. Data are public.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Experimental settings are presented in the main paper and the appendix.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: All LLM decodings in the paper are greedy decoding. There is little randomness in the evaluation process. Evaluating training randomness exceeds our current compute resource capability.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).

- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Computer resource is disclosed in appendix.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The authors follow NeurIPS Code of Ethics.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: Broader impacts are discussed in the appendix.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.

- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper poses no such risks.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [N/A]

Justification: The paper does not use existing assets other than standard python libraries or public models.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.

- If this information is not available online, the authors are encouraged to reach out to the asset’s creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [N/A]

Justification: The paper does not release new assets.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: he paper does not involve crowdsourcing nor research with human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The paper does not pose such risks.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

1032 Question: Does the paper describe the usage of LLMs if it is an important, original, or
1033 non-standard component of the core methods in this research? Note that if the LLM is used
1034 only for writing, editing, or formatting purposes and does *not* impact the core methodology,
1035 scientific rigor, or originality of the research, declaration is not required.

1036 Answer: [N/A]

1037 Justification: The core method development in this research does not involve LLMs as any
1038 important, original, or non-standard components, other than using LLM judge for sentiment
1039 evaluation.

1040 Guidelines:

- 1041 • The answer [N/A] means that the core method development in this research does not
1042 involve LLMs as any important, original, or non-standard components.
- 1043 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not
1044 be described.